INASS

International Journal of
Intelligent Engineering & Systems

http://www.inass.org/

Adaptive Web Penetration Testing with Q-Learning and Deep Q-Networks on OWASP Juice Shop

Kevin Imanuel Setiawan

Bachelor's Program in Information Technology, Telkom University, Surabaya, Indonesia

* Corresponding author: kevinit@student.telkomuniversity.ac.id

Abstract: This study compares tabular Q-learning (QL) and a deep Q-network (DQN) for adaptive web penetration testing on OWASP Juice Shop. Both agents operated under a fixed budget of 1,585 episodes over 120 endpoint-payload actions. Candidate findings were quality-gated using control-differential probes, while cross-site scripting candidates also required execution in a browser. QL produced 14 verified findings and DQN produced 12; their first findings appeared in episode 1, and each reached its final observed finding count by episodes 787 and 443, respectively. A separate detector benchmark yielded 13 true positives, no false positives, and one false negative among labeled actions, with 46 actions left unknown. A scoped comparison on two shared endpoints recorded 10 QL findings and two relevant ZAP findings, but the request procedures were not equivalent. The results demonstrate feasibility in this environment, not generalization to other applications.

Keywords: web penetration testing; reinforcement learning; Q-learning; deep Q-network; OWASP Juice Shop; OWASP ZAP

1. INTRODUCTION

Web applications expose multiple input points and behaviors, so penetration testers must choose requests under limited time and request budgets. Reinforcement learning offers a way to learn action preferences from observed responses. Q-learning (QL) estimates action values in a table, whereas a deep Q-network (DQN) approximates values with a neural network [1]–[3]. Prior research has explored reinforcement learning for penetration testing and capture-the-flag environments, while emphasizing trade-offs between model-free learning and prior knowledge [4].

Earlier research frames penetration testing as a sequence of decisions in which observations guide the next probe. Model-free reinforcement learning is attractive because an agent can update action preferences through interaction without requiring a complete model of the target. Tabular Q-learning estimates action values directly for state-action pairs [1], [2], whereas deep Q-networks approximate action values with a neural network for richer representations [3]. Neither approach removes the need to define what actions are possible, how responses are represented, how rewards are assigned, and what evidence qualifies as a finding. Those design choices shape both the learned behavior and the claims that can be drawn from an experiment.

Research applying reinforcement learning to penetration testing and capture-the-flag tasks has highlighted the trade-off between model-free exploration and a priori knowledge [4]. A predefined endpoint and payload library improves repeatability and makes an experiment computationally manageable, but it narrows the search problem. This study declares the human-selected endpoints, payload classes, and endpoint surfaces as prior knowledge. The policy adapts its

selection among the predefined requests; it does not discover arbitrary URLs, crawl the application, generate new payloads, or conduct unrestricted autonomous testing.

This study evaluates QL and DQN on OWASP Juice Shop, a deliberately vulnerable web application for security training [5]. We compare verified findings under a fixed episode budget and describe a scoped QL comparison with OWASP ZAP. Findings pass control-differential checks; XSS candidates additionally require browser execution. The evaluation is specific to the target and action set and does not establish generalization to arbitrary websites.

The evaluation addresses three descriptive questions: how many findings pass the verification gate for QL and DQN under the same episode budget; how their finding categories, endpoint activity, reward totals, and final-discovery episodes differ in the recorded runs; and what can be inferred from a two-endpoint comparison with ZAP when scan workflows remain different. The experiment does not test statistical significance or transfer to unseen applications.

2. MATERIALS AND METHODS

2.1 Test Environment and Action Space

The target was OWASP Juice Shop version 20.2.0 running locally. Eight configured endpoints and 15 payloads formed 120 endpoint-payload actions: six SQL injection (SQLi), four cross-site scripting (XSS), and five fuzzing probes. Endpoint surface labels and payloads were manually curated from knowledge of the application. This human prior narrows the search space, so the system is not fully autonomous. The state comprises HTTP status, response length, error, reflection and vulnerability indicators, endpoint index, response-time category, and content type. DQN receives a 28-dimensional one-hot encoding.

The eight endpoint configurations cover product search, file access, product reviews, order tracking, user login, feedback, and challenge retrieval. GET probes are placed in the configured query or path; login, review updates, and feedback use structured request bodies. The feedback operation also obtains a captcha through the application endpoint and supplies required fields. These endpoint-specific request formats reduce the chance that malformed traffic is mistaken for evidence of security, while keeping the experiment within the implemented API flows.

TABLE 1. ENDPOINTS AND INPUT SURFACES

Index	Method and path	Surface
0	GET /rest/products/search?q=	SQL
1	GET /ftp/	File
2	GET /rest/products/1/reviews	Plain
3	GET /rest/track-order/	Plain
4	POST /rest/user/login	SQL
5	PUT /rest/products/1/reviews	Plain
6	POST /api/Feedbacks	Plain
7	GET /api/Challenges	Plain

The action space is the Cartesian product of eight endpoint indices and 15 payload indices. An action index maps to endpoint = floor(action/15) and payload = action modulo 15. The 15 payload templates comprise six SQL injection probes, four cross-site scripting probes, and five fuzzing probes. The action set is fixed before training. The agent does not create new endpoints, infer unlisted routes, mutate payload syntax, or expand the payload library. It adapts only by choosing among these 120 predefined endpoint-payload pairs in response to the recorded state and reward.

The state summarizes the latest response and its request location. It contains HTTP status category, response-length category, error flag, reflection flag, vulnerability indicator, endpoint index, response-time category, and content-type category. The representation is not a full application model: separate server-side conditions may map to the same tuple if their recorded features coincide. For DQN, categorical fields are encoded as one-hot vectors: five status values, four response-length categories, eight endpoints, three response-time categories, and five content types. The three remaining indicators are scalar inputs. The resulting DQN input dimension is 28; QL stores the discrete state tuple as a key.

TABLE 2. OBSERVED STATE COMPONENTS

Field	Encoding	Role
HTTP status	5 categories	Response class
Response length	4 categories	Size category
Error	Scalar flag	Request/server error evidence
Reflection	Scalar flag	Input-reflection evidence
Vulnerability	Scalar indicator	Analyzer

		classification
Endpoint	8 categories	Request location
Response time	3 categories	Latency category
Content type	5 categories	Response format

2.2 Agent Configuration and Training Protocol

Each agent used 1,585 episodes and random seed 42. QL used learning rate 0.1, discount factor 0.9, epsilon from 1.0 to 0.1 with decay 0.90, and optimistic initial values of 5. DQN used a 64-unit hidden layer, learning rate 0.001, batch size 32, replay capacity 10,000, target update interval 10, and epsilon decay 0.9965. Exploration schedules differed to avoid DQN reaching its exploration floor too early. There was no early stopping. Only verified vulnerable actions were masked; previously attempted or failed actions remained available. A benign hello probe established a separate response baseline for each endpoint. Rewards were +50 for verified findings, +10 for anomalies or inconclusive candidates, −1 for normal responses, and −5 for errors, with a configured response-length bonus of 0, 0, 5, or 10.

TABLE 3. AGENT CONFIGURATION

Parameter	Q-learning	DQN
Episode budget	1,585	1,585
Learning rate	0.1	0.001
Discount factor	0.9	0.9
Exploration	ε: 1.0 to 0.1; decay 0.90	ε: 1.0 to 0.1; decay 0.9965
Value initialization	5.0	Neural network
Hidden units	—	64
Batch / replay	—	32 / 10,000
Target update	—	Every 10 updates
Random seed	42	42

The two methods use the same environment and observations but represent action values differently. QL updates a discrete table according to $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$, where s and s′ denote the current and next states, a is the selected action, r is its reward, α is the learning rate, and γ discounts future value [1], [2]. An epsilon-greedy policy samples an action for exploration with probability ε and otherwise selects a high-valued action. Optimistic initialization assigns unvisited QL values a starting estimate of 5.0.

DQN approximates action values with a neural network and learns from stored transitions with a target network. The configured hidden dimension is 64, learning rate is 0.001, batch size is 32, replay capacity is 10,000, and target update interval is 10. The DQN epsilon decay of 0.9965 is more gradual than QL's 0.90. It was selected to preserve exploration longer under random initial network values. Thus, the comparison evaluates each algorithm with an appropriate configured schedule; it does not isolate network architecture while holding all hyperparameters numerically identical.

The reward function distinguishes accepted findings, anomalies, normal responses, and request errors. Verified findings receive +50; anomalies or inconclusive candidates receive +10; normal responses receive −1; and errors receive −5. A response-length bonus adds 0, 0, +5, or +10 according to the configured response-length category. The bonus can influence total reward but does not change the verification rule used to count a vulnerability.

TABLE 4. REWARD COMPONENTS

Outcome	Reward	Purpose
Verified vulnerability	+50	Positive signal after verification
Anomaly/inconclusive	+10	Signal without verified-vulnerability claim
Normal response	−1	Small negative signal
Error response	−5	Penalty for failed request
Length bonus	0, 0, +5, +10	Category-based additional reward

The training protocol follows a repeatable sequence. Before training, the client sends the benign value "hello" to each endpoint and records a separate baseline. During each episode the policy selects one action, the HTTP client sends it, and the response analyzer generates the next state and candidate label. Any candidate then passes through its applicable verification gate. The final verified, anomaly, normal, or error result determines the reward; the transition is recorded and the agent updates its values before the next episode. This order separates baseline collection, detection, verification, and learning.

Action masking applies only to actions already confirmed as vulnerable. Failed, normal, previously attempted, or rejected actions remain selectable. There is no exhausted-action masking and no early stop after the agent has encountered its currently known findings. Each agent therefore

completes all 1,585 episodes. This fixed budget supports comparison of complete trajectories, although it also means requests continue after the final new finding in a run.

2.3 Finding Verification and Detector Benchmark

HTTP candidates were checked against a benign control and, when applicable, a neutralized-payload control; a finding was accepted only if neither reproduced its vulnerability class. Connection errors did not count as verification. XSS was evaluated in headless Chromium on the corresponding single-page application route; acceptance required execution of an instrumented canary, with a warmup navigation and bounded polling. A separate detector benchmark evaluated all 120 actions against curated labels. It used the browser path for XSS and excluded 46 unknown actions from binary metrics. Because ground-truth confirmation and detector evidence partly share mechanisms, this is not a fully independent oracle.

The analyzer also applied endpoint-surface constraints when assigning vulnerability classes. SQLi evidence required a database-error response or a sufficiently changed successful response relative to the endpoint baseline. File-disclosure labels were restricted to the configured file surface and required recognizable content or the configured extension-bypass evidence. Fuzzing findings required an echoed probe in a server-error response. These constraints reduce generic error-page and reflection-only findings, but their accuracy remains bounded by the configured signatures and curated surface labels.

Response classification uses evidence tied to the endpoint surface. SQLi requires database-error evidence or a sufficiently changed successful response relative to that endpoint's baseline. File-disclosure findings are restricted to the configured file surface and require recognizable file content or the specified extension-bypass evidence. Fuzzing findings require a server-error response that echoes the probe. Generic error pages, directory listings, and reflection alone do not qualify. These rules reduce broad heuristic matches but remain dependent on implementation logic and the human-configured surfaces.

The control-differential step addresses false positives by asking whether a benign request or a less exploit-like variant produces the same evidence. Payloads for which neutralization would make no meaningful change used only the benign control. An inconclusive candidate was retained as an anomaly reward rather than counted as a verified vulnerability. The browser check serves a separate purpose: it distinguishes data reflected in an HTTP response from a script that actually executes in the rendered application.

HTTP candidates are checked against a benign control and, when neutralization meaningfully changes the candidate, a neutralized-payload control. A finding is accepted only if neither control reproduces the same vulnerability class. Payloads whose neutralization would be a no-op use only the benign control; connection errors do not count as verification. Inconclusive candidates remain anomalies rather than vulnerabilities. The differential checks improve evidence quality at the cost of additional requests after a candidate appears.

XSS uses a separate browser-execution check because JavaScript reflected in an HTTP response does not necessarily execute in the application. Headless Chromium loads the corresponding single-page application route with an instrumented vector that sets a canary. The system accepts an XSS finding only when the canary executes in the DOM. A warmup navigation loads application assets before probes, and bounded polling checks for canary execution. This confirms execution for the configured routes and vectors; it does not cover every browser, route, or client-side context.

2.4 ZAP Comparison and Metrics

The controlled ZAP comparison was restricted to the two endpoints shared with QL: product search and the FTP surface. ZAP provides automated scanning and alert reporting [6]. We report verified finding count, hit rate (findings divided by training episodes), payload requests per finding, first-finding episode, episode at which the final observed finding count was reached, and unique endpoint–class locations. These are in-sample training results: no held-out test set was used. The endpoint scope is shared, but scan strategy, payload selection, verification overhead, and request accounting differ between QL and ZAP.

A finding count and a unique vulnerable location are also different measures. The unique-location

measure counts distinct endpoint-index and vulnerability-class pairs. Several accepted payload-level findings at the same endpoint and class can contribute multiple findings but one location. This distinction prevents payload variants from being presented as separate vulnerable application locations.

The 1,585 episodes are the learning budget for each agent. The experiment also records baseline probes, verification probes, and total requests including verification. Episode count, selected payload-request count, and end-to-end HTTP transaction count therefore represent different quantities. Request-per-finding ratios must be read with their counting rule, particularly because control probes are issued only after candidates are detected.

3. RESULTS AND DISCUSSION

3.1 QL and DQN Training Results

QL produced 14 verified findings and DQN produced 12 (Table 5). Both found a vulnerability in episode 1. QL reached its final observed count by episode 787, whereas DQN did so by episode 443. These values describe accumulation during the recorded run and do not establish convergence or unseen-target performance. QL had more findings, while DQN accumulated a larger total reward, which reflects reward across all episodes rather than finding count alone.

TABLE 5. TRAINING OUTCOMES REPORTED IN THE THESIS

Measure	QL	DQN
Episodes	1,585	1,585
Verified findings	14	12
Categories	6 SQLi; 5 XSS; 3 fuzz/file	6 SQLi; 4 XSS; 2 fuzz/file
Hit rate	0.88%	0.76%
Requests/finding	113.21	132.08
First finding	Episode 1	Episode 1
Final count reached	Episode 787	Episode 443
Unique locations	5	5
Elapsed time	154.23 s	160.04 s
Total reward	3,785	10,209

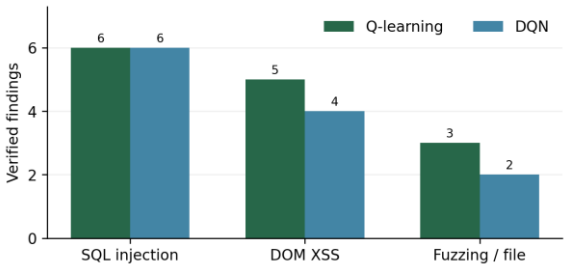
The category counts show six SQLi findings for each agent. QL reported one additional XSS and one additional fuzzing/file finding. Multiple payload findings at a single endpoint and class contribute one unique location; these results are not an inventory of independent production vulnerabilities.

Both agents reported six SQLi findings. QL's two-finding advantage is accounted for by one additional XSS and one additional fuzzing/file finding. The counts do not measure severity, exploitability in a production context, or remediation priority; the experiment reports verified class outcomes rather than a risk-weighted score. The figure visualizes category counts to make this composition difference visible without implying that the categories have equal operational impact.

The first finding appeared at episode 1 for both agents, so this measure does not distinguish their initial discovery in the recorded runs. DQN reached its own final finding count by episode 443, before QL reached its final count by episode 787. This does not mean DQN found more issues: the count measures total accepted findings, whereas the episode measures when the run reached that count. A single seeded trajectory cannot show whether either pattern is stable across runs.

The cumulative reward should not be interpreted as a direct vulnerability-coverage metric. It aggregates rewards for every episode, including anomalies, normal responses, errors, and length bonuses. DQN can therefore earn a higher total reward even while reporting fewer verified findings. Future comparisons should report reward and verified-finding curves together and summarize variation over repeated seeds.

Fig. 1. Verified finding categories reported for each agent.



3.2 QL Endpoint Activity

Product search accounted for most QL requests and findings. The FTP surface produced three findings from 39 requests; track-order produced three; login produced one. Other configured endpoints produced no verified findings in this run. The distribution reflects the curated action space and should not be interpreted as a complete vulnerability inventory.

The endpoint request counts sum to the 1,585 episode actions. Product search accounts for 1,184 requests, or roughly three quarters of the budget, while track-order accounts for 252. The other six endpoints together account for 149 requests. The learned policy therefore concentrated its actions at search and track-order, especially search. This describes policy allocation in the run; it does not demonstrate that the remaining endpoints were exhaustively tested outside the configured payload set.

The endpoint finding counts sum to 14: seven for search, three for FTP, three for track-order, and one for login. The report separately gives five unique vulnerable locations because multiple accepted payload findings can correspond to the same endpoint-class pair. Alert consolidation rules vary between tools, so any cross-tool comparison must state whether it counts payload-level findings, unique locations, or consolidated alerts.

The difference between reward and verified-finding totals follows from the reward design. A vulnerability contributes a large positive reward, but the final cumulative score also includes normal responses, errors, anomalies, and response-length bonuses throughout all episodes. DQN can therefore obtain a higher cumulative reward while producing fewer distinct accepted findings. Reward should be interpreted as an optimization signal used by each agent, not as a direct substitute for vulnerability coverage.

TABLE 6. QL REQUESTS AND VERIFIED FINDINGS BY ENDPOINT

Endpoint	Requests	Findings
Product search	1,184	7
FTP surface	39	3
Product reviews (GET)	24	0
Track order	252	3
User login	24	1
Product reviews (PUT)	21	0
Feedback	22	0
Challenges	19	0
Total	1,585	14

3.3 Detector Benchmark

The labeled benchmark contained 13 true positives, no false positives, one false negative, and 60 true negatives; 46 actions remained unknown. Precision was 1.000, recall 0.929, F1 0.963, and specificity 1.000. The false negative was the SVG XSS action on the track-order route. Zero false positives applies only to the 60 labeled negatives and does not establish perfect performance on untested applications or payloads.

For clarity, precision is TP/(TP+FP), recall is TP/(TP+FN), F1 is the harmonic mean of precision and recall, and specificity is TN/(TN+FP). Here these measures are computed only on the benchmark's labeled subset. The 46 unknown actions are not assumed safe and are not included as negatives; this prevents unknown labels from artificially increasing specificity.

Precision is TP/(TP+FP), recall is TP/(TP+FN), F1 is the harmonic mean of precision and recall, and specificity is TN/(TN+FP). These values are computed only for labeled actions. The 46 unknown actions are not assumed safe and are excluded from the negative class; recoding them as negatives would inflate specificity without evidence. The one false negative, an SVG XSS action on the track-order route, also shows that the configured detector and verification path did not recover every curated positive.

No false positives among 60 labeled negative actions is useful evidence that the gates suppressed several noisy response patterns in this benchmark. It is not a guarantee of zero false positives in future tests: the labeled set is finite, labels are curated, and the oracle partly shares the detector's evidence mechanisms. A stronger independent assessment would have blinded reviewers adjudicate browser and server evidence without seeing detector output or agent identity.

TABLE 7. DETECTOR PERFORMANCE ON LABELED ACTIONS

Metric	Value
True positives	13
False positives	0
False negatives	1
True negatives	60
Unknown actions	46
Precision	1.000
Recall	0.929
F1-score	0.963
Specificity	1.000

3.4 Scoped Comparison with OWASP ZAP

For the two shared endpoints, the thesis reports 1,223 QL payload requests for 10 findings (122.30 requests per finding). The controlled ZAP scan reports 1,585 requests and two relevant findings (792.50 requests per finding). The ratio of these rates is approximately 6.48. ZAP produced 32 raw alerts, of which two were treated as relevant. This is endpoint-scope matching, not a fully procedure-matched apples-to-apples comparison: request workflow, payloads, verification overhead, and counting rules differ. The 6.48 ratio is descriptive for this experiment and cannot support a general efficiency claim.

Restricting both tools to product search and FTP makes the target scope more comparable than comparing QL's eight-endpoint run with a broader scan. Still, QL selects individual payload actions through its policy, while ZAP follows its own spidering and active-scan workflow. They can issue different request types and volumes, apply different payloads, and use different alert-consolidation and acceptance criteria. QL verification probes and the ZAP workflow also contribute differently to the request totals.

The ratio 792.50/122.30 is approximately 6.48, but it is descriptive rather than a normalized effect estimate. It does not control for scan duration, payload complexity, severity, manual review cost, or the extra baseline and verification transactions. The raw request and finding counts should be reported beside the ratio. A stronger comparison would align the endpoint allowlist, authentication state, request and time budgets, payload families, request accounting, alert definitions, and independent manual confirmation.

TABLE 8. COMPARISON ON TWO SHARED ENDPOINTS

Method	Requests	Relevant findings	Requests/finding
QL selected-payload subset	1,223	10	122.30
Controlled ZAP scan	1,585	2	792.50

3.5 Limitations

The evaluation used one target and one seed per algorithm; training results are in-sample. Endpoint labels and payloads encode human knowledge. The detector benchmark shares evidence with its verification oracle, and the ZAP comparison does not align all scan procedures. Future work should repeat runs across seeds, use held-out targets or actions, obtain independent manual validation, and equalize request and time budgets for baseline comparisons.

3.6 Interpretation of the QL–DQN Difference

The finite action set contains 120 choices and the state representation is compact and engineered. Tabular QL is consequently a reasonable baseline because values can be stored for discrete observed states without function approximation. DQN may be useful when state representations grow or share structure, but this experiment does not establish that advantage. The finding totals and reward totals are outcomes of one run under different exploration schedules; they should be treated as an implementation comparison rather than a general ranking of learning algorithms.

3.7 Threats to Validity

Construct validity is limited by the operational definition of a finding: a candidate must match the analyzer rules and pass its verification gate. The evaluation does not assign severity or measure business impact. Internal validity is limited by a single random seed per agent and algorithm-specific hyperparameters. The observed differences may reflect trajectory randomness and configuration choices as well as the learning algorithm.

External validity is limited to one version of one intentionally vulnerable application, one local deployment, and one curated payload library. Endpoint surfaces and payload choices encode human knowledge. The test does not cover changing production state, real authentication workflows, rate limits, network variability, or a broad range of web frameworks. A fixed seed improves reproducibility but cannot estimate run-to-run variation.

Comparison validity is limited because ZAP and the RL agents use different scan workflows and request-count rules. Equal endpoint scope alone does not ensure equal opportunities to find vulnerabilities. Consolidation rules and evidence thresholds also affect the finding numerator. These limitations constrain the interpretation of the 6.48 ratio and preclude a general efficiency claim.

3.8 Follow-up Experiments

Future work should run each method across multiple seeds and report mean, spread, confidence intervals, and per-seed discovery curves. A held-out application or endpoint set would test transfer, provided the held-out labels are not used to design the action space or detection rules. Ablations could remove surface gating, browser verification, or optimistic initialization to estimate how each design choice affects discovery and false positives.

For a stronger ZAP comparison, both systems should use a published endpoint allowlist and aligned authentication state. Request budgets and wall-clock budgets should be reported separately, since equalizing only one can favor a system with more expensive verification. Candidate findings should be deduplicated by endpoint, vulnerability class, and evidence, then manually reviewed by an assessor blinded to tool identity. Coverage, request cost, time-to-first-verified-finding, and false-positive review burden can then be compared under a shared protocol.

4. CONCLUSION

Under 1,585 episodes on OWASP Juice Shop, QL obtained 14 verified findings and DQN obtained 12. DQN reached its final observed finding count earlier, while accumulating greater total reward. The detector benchmark agreed strongly with curated labels, subject to partial dependence on shared verification evidence. The scoped ZAP comparison favored QL in requests per finding, but procedural differences make that result descriptive. Repeated and held-out evaluations are required before drawing broader conclusions.

CONFLICT OF INTEREST

The author declares no conflict of interest. (Please confirm this declaration before submission.)

REFERENCES

[1] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992, doi: 10.1007/BF00992698.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[3] V. Mnih et al., “Human-level control through deep reinforcement learning,” Nature, vol. 518, pp. 529–533, 2015, doi: 10.1038/nature14236.

[4] F. M. Zennaro and L. Erdodi, “Modeling penetration testing with reinforcement learning using capture-the-flag challenges: Trade-offs between model-free learning and a priori knowledge,” arXiv:2005.12632, 2020.

[5] OWASP Foundation, “OWASP Juice Shop.” [Online]. Available: https://owasp.org/www-project-juice-shop/. Accessed: Oct. 9, 2026.

[6] OWASP Foundation, “ZAP Automation Framework.” [Online]. Available: https://www.zaproxy.org/docs/desktop/addons/automation-framework/. Accessed: Oct. 9, 2026.